

Flujo de Trabajo Técnico Automatizado para Creadores de Contenido Audiovisual

Introducción: Este informe describe un flujo de trabajo **completo y automatizado** para un creador de contenido que utiliza Adobe Premiere Pro y Photoshop, integrando herramientas en Python y React sobre Windows. Se cubren todas las etapas del proceso, desde la descarga de VODs de plataformas de streaming (Twitch, YouTube, Kick), la generación asistida por IA de miniaturas, la subida automatizada de videos a YouTube con metadatos (título, descripción, etiquetas) y programación de publicación, la creación de subtítulos (en español y traducidos), hasta el análisis de rendimiento de los videos (CTR, retención, SEO, etc.). A lo largo del informe se recomiendan herramientas, bibliotecas y se proporcionan comparativas, con ejemplos de implementación cuando es posible.

1. Descarga Automatizada de VODs (Twitch, YouTube, Kick)

Para obtener videos de transmisiones pasadas (VODs) de Twitch, YouTube y Kick de forma programática, es necesario recurrir a herramientas de descarga de videos. A continuación se presentan las principales opciones y consideraciones para cada plataforma:

- **YouTube:** La forma más común es usar **youtube-dl** o su fork actualizado **yt-dlp**. Estas utilidades de línea de comandos permiten descargar videos de YouTube (y cientos de sitios) fácilmente, y cuentan con opciones para seleccionar la calidad de video, descargar listas de reproducción completas, subtítulos, etc ¹ ². **yt-dlp** es altamente recomendable por ser más actualizado; es un programa de consola que también puede integrarse en Python importándolo como librería (`pip install yt-dlp`).
- **Twitch:** Twitch no ofrece una API oficial para descargar VODs directamente (no existe un endpoint que devuelva un MP4 o similar) ³. Por ello, se utilizan métodos indirectos:
- **yt-dlp:** También soporta descargas de Twitch VOD (apuntando a la URL del VOD). Internamente, recupera la playlist HLS (`.m3u8`) del video y descarga los segmentos de streaming.
- **Herramientas especializadas:** Por ejemplo, *twitch-dl* es una utilidad CLI dedicada a Twitch que mejora la velocidad usando conexiones concurrentes ⁴. Otro proyecto popular es *Streamlink*, que puede extraer y grabar streams de Twitch (en vivo o VOD) usando FFmpeg.
- **Nota:** Si el VOD de Twitch es de suscriptor o privado, se requerirá autenticación (OAuth) y aún así Twitch podría bloquear la descarga. En general, para VODs públicos, las herramientas mencionadas funcionan bien.
- **Kick:** Kick es una plataforma relativamente nueva y actualmente no tiene herramientas tan maduras como Twitch/YouTube. Sin embargo, es posible descargar VODs de Kick extrayendo la URL HLS de la página (archivo `master.m3u8`) y descargando los segmentos:
- **yt-dlp + FFmpeg:** Según la comunidad, yt-dlp en conjunto con gestores de descargas como *aria2* puede descargar VODs de Kick rápidamente ⁵ ⁶. El flujo es obtener la URL del manifest HLS y luego dejar que yt-dlp descargue y combine los fragmentos.

- **Proyectos dedicados:** Existen herramientas open-source específicas, como *Kick Downloader* (por ejemplo, **Kick VOD Downloader** escrito en Go), que automatizan la obtención del playlist HLS y la conversión a MP4 ⁷. También hay utilidades GUI (por ejemplo extensiones de navegador y páginas web) que permiten pegar la URL del VOD de Kick y obtener un archivo, aunque para integraciones programáticas es preferible usar un script propio con yt-dlp o una librería HTTP.

A modo de resumen, la siguiente tabla compara algunas herramientas para la descarga de VODs:

Herramienta	Plataformas Soportadas	Características Relevantes	Uso Programático
<code>yt-dlp</code> (fork de youtube-dl)	YouTube, Twitch, otros 1000+ sitios, incluye Kick (vía HLS)	CLI multiplataforma. Permite elegir formato/calidad, descargar playlists, subtítulos, etc ² . Muy activa en parches y nuevas plataformas.	Sí – ejecutar vía comando o usar como librería Python (<code>yt_dlp</code>).
<code>twitch-dl</code>	Twitch (VODs y clips)	Descarga VODs de Twitch aprovechando <i>múltiples conexiones</i> para acelerar la bajada ⁴ . Listar videos/clips de un canal.	Sí – es una herramienta CLI (escrita en Python) fácil de invocar desde scripts.
<code>Streamlink</code>	Twitch, YouTube (y otros servicios streaming soportados)	Enfocado en extraer streams y canalizarlos a un reproductor o archivo. Ideal para capturar transmisiones en vivo o VODs usando FFmpeg.	Sí – se invoca por CLI (<code>streamlink <url> best -o file.mp4</code>). También tiene API Python limitada.
<code>Kick VOD Downloader</code>	Kick	Herramienta especializada (ej. en Go) para descargar VODs de Kick ⁷ . Automatiza la obtención del <code>.m3u8</code> y combina segmentos en MP4.	Parcial – al ser standalone, se puede llamar vía CLI. O usar su código fuente como referencia para implementar en Python (requests + ffmpeg).
Twitch API (oficial)	Twitch (teóricamente)	<i>Nota:</i> La API de Twitch no provee un método directo para descargar VODs por razones técnicas y de políticas ³ . Solo permite obtener IDs, URLs de reproducción (HLS) y metadatos.	No disponible para descarga directa – se debe complementar con descarga HLS manual.

Ejemplo de Implementación: Usando `yt-dlp` en Python para descargar un VOD de YouTube o Twitch. Primero, instalar la librería: `pip install yt-dlp`. Luego, en un script Python:

```
import yt_dlp

url = "https://www.youtube.com/watch?v=VIDEO_ID" # o Twitch VOD URL
ydl_opts = {"outtmpl": "video.%(ext)s"} # guardar con nombre básico
with yt_dlp.YoutubeDL(ydl_opts) as ydl:
    ydl.download([url])
```

Este snippet descargará el video en la calidad por defecto más alta. De igual forma, se puede usar para Twitch VODs (yt-dlp detectará automáticamente la plataforma y resolverá el HLS). Para Kick, dado que aún no tiene extractor nativo, se podría primero obtener la URL HLS (como muestra el método manual de abrir DevTools y buscar `master.m3u8` ⁸) y luego pasar esa URL directamente a yt-dlp.

2. Generación de Miniaturas con Inteligencia Artificial

Crear una miniatura (thumbnail) atractiva es crucial para el éxito de un video, pero hacerlo manualmente en Photoshop puede ser tardado. Aquí proponemos **soluciones basadas en IA** para agilizar o automatizar la generación de miniaturas, manteniendo calidad profesional:

- **Modelos Generativos de Imágenes:** Herramientas de IA como *Stable Diffusion* o *DALL·E* permiten generar imágenes a partir de descripciones de texto. Por ejemplo, se podría describir la temática del video y pedir a Stable Diffusion que genere una imagen de fondo llamativa. Stable Diffusion es open-source, se puede ejecutar localmente (requiere GPU) o usar a través de APIs/servicios en la nube. Hay proyectos que combinan Stable Diffusion con otros elementos para crear thumbnails: un ejemplo usa Stable Diffusion y un modelo de lenguaje (LLM) para proponer conceptos y montar la miniatura ⁹ ¹⁰. Estas soluciones pueden *idear* conceptos, generar gráficos, remover fondos automáticamente y añadir texto, todo sin intervención manual.
- **Herramientas online especializadas:** Existen aplicaciones web con IA enfocadas en miniaturas (p. ej. Canva con IA, Simplified, Fliki). Por ejemplo, Fliki ofrece un “*creador de miniaturas con IA*” que promete generar miniaturas personalizables con imágenes de alta calidad para maximizar el CTR ¹¹. Estas plataformas a veces ofrecen APIs o al menos automatizaciones mediante sus SDK. Suelen combinar generación de imágenes, plantillas prediseñadas y opciones de edición rápida (fuentes, colores, superposiciones). La desventaja es que pueden requerir pago o intervención manual si no tienen API abierta.
- **Detección de rostros y fotogramas del video:** Un enfoque práctico es **extraer fotogramas** del propio video que sean candidatos a miniatura. Usando Python con bibliotecas de visión (OpenCV, por ejemplo), se puede analizar el video para encontrar frames con ciertas características: rostros bien visibles (usando detección de rostros), escenas con alta emoción o acción, texto legible en pantalla, etc. Por ejemplo, detectar automáticamente dónde aparece la cara del streamer con buena expresión, y tomar esa imagen. Una vez elegido el frame, se puede mejorar:
- **Mejora de imagen:** Aplicar *upscaling* (super resolución) mediante IA para mejorar la nitidez (p. ej. ESRGAN o realesrgan para escalar a 4K).
- **Edición automatizada en Photoshop:** Mediante scripting (ver sección 6) se podría automatizar la inserción de ese frame en una plantilla de Photoshop donde quizás ya haya un diseño (logo,

marco, etc.), agregar texto (el título del video, por ejemplo) con fuentes corporativas, y exportar la miniatura final.

- **Modelos de evaluación:** Opcionalmente, se puede emplear IA para *predecir* qué tan efectiva podría ser una miniatura. Por ejemplo, algunos investigadores han usado redes neuronales para evaluar la estética o si la miniatura destaca. No es una ciencia exacta, pero podría servir para elegir entre varias opciones generadas.

A continuación, se comparan algunas opciones para la creación de miniaturas con IA:

Enfoque / Herramienta	Descripción	Ventajas	Desventajas
Stable Diffusion (Difusores)	Modelo generativo de imágenes a partir de texto (o imágenes). Se puede ejecutar localmente o usar via API (Replicate, etc.). Ej: generar fondo temático (paisaje, gráfico abstracto, etc.).	Control total sobre la generación; sin costo por uso local. Puedes entrenar/ajustar si requieres un estilo específico.	Requiere recursos (GPU potente) y conocimientos para obtener buenos resultados. No garantiza consistencia con branding (debe guiarse mediante prompts).
DALL-E 2/3 (OpenAI)	API de OpenAI para generar imágenes por texto.	Muy buena calidad y coherencia en muchos casos; rápido de implementar vía API.	Costo por uso (pago por imagen generada). Algunas restricciones de contenido. Sin control absoluto sobre estilo específico de marca.
Servicios IA de Thumbnails (Canva, Fliki, etc.)	Plataformas web con flujo simplificado: generan miniaturas con IA + plantilla. P. ej. Fliki permite introducir el título y elige o crea una imagen, facilitando agregar texto y formas ¹² ¹³ .	Muy fáciles para usuarios no técnicos. Rápidas (plantillas en segundos). Integran banco de imágenes y tipografías.	Pueden requerir intervención manual o suscripción. La automatización completa depende de si ofrecen API. Menos flexibles en personalización avanzada.
Selección de fotogramas + Photoshop	Estrategia casera: elegir automáticamente un fotograma del video (usando detección de rostros, análisis de color/ movimiento) y luego usar un script de Photoshop para componer la miniatura (añadir texto, logos).	Asegura relevancia (usa imagen real del video). Photoshop permite resultado profesional y consistente con branding. Todo el proceso se puede automatizar con scripting.	Puede requerir afinar la lógica de selección de frame. La calidad del frame puede ser baja (solución: aplicar IA upscale). Scripting de Photoshop tiene curva de aprendizaje.

Ejemplo de flujo de generación: Supongamos un video de gameplay con momentos destacados: 1. Usar Python/OpenCV para escanear el video y detectar frames donde aparece la cara del streamer sonriendo o con gesto de sorpresa. También verificar brillo/contraste del frame. 2. Tomar 3-5 frames candidatos y pasarlos por un modelo de super-resolución para mejorarlos. 3. Con cada frame candidato, usar la API de *remove.bg* (o `rembg` localmente) para eliminar el fondo, dejando solo al streamer recortado. 4. Usar Stable Diffusion para generar un fondo dinámico relacionado al juego (por ejemplo, "explosión de color con elementos de <nombre del juego>"). 5. Combinar el recorte del streamer sobre el fondo generado, y añadir un texto llamativo (quizá el título del highlight) con una fuente legible. Esto podría hacerse automáticamente con un script de Photoshop: la biblioteca **Photoshop Python API** (ver sección 6) permite crear documentos, insertar la imagen de fondo, luego la del personaje y una capa de texto, aplicando estilos ¹⁴ ¹⁵. 6. Exportar la imagen final a JPG/PNG con la resolución requerida (1280x720 px para miniaturas de YouTube).

Este proceso podría generar varias variantes y luego un creador podría elegir la mejor, o incluso ese paso de elección podría asistirse con un modelo que evalúe cuál es más *eye-catching*.

En resumen, combinar IA generativa para acelerar la **idea y creación** de miniaturas puede ahorrar mucho tiempo y aportar creatividad (probar diseños que manualmente no se hubieran ocurrido), manteniendo la calidad con herramientas de edición tradicionales para el toque final.

3. Automatización con las APIs de YouTube (Subida, Metadatos, Programación)

YouTube ofrece una API oficial (YouTube Data API v3) que permite realizar prácticamente todas las acciones que se hacen en YouTube Studio de forma automatizada. Para nuestro flujo de trabajo, las capacidades más relevantes son:

- **Subir videos por API:** Usando el endpoint `videos.insert` es posible subir un archivo de video. Se debe autenticar la solicitud con OAuth 2.0 (las credenciales del canal de YouTube del creador). Google provee librerías cliente (por ejemplo, **google-api-python-client** en Python) que simplifican realizar la solicitud de subida. En la documentación oficial hay ejemplos de scripts en Python para subir un video, estableciendo campos como título, descripción, etiquetas y categoría ¹⁶ ¹⁷.
- *Ejemplo:* El siguiente comando simulado muestra cómo el script de ejemplo de Google subiría un video:

```
python upload_video.py --file="video_editado.mp4" \  
  --title="Mi Video Editado" \  
  --description="Resumen de stream jugando X..." \  
  --keywords="juego X, gameplay, highlights" \  
  --category="20" \  
  --privacyStatus="private"
```

Esto subiría el video con estado privado inicialmente (útil para luego programar publicación). En Python, usando la librería, se construiría un recurso `Video` y se llamaría al método `youtube.videos().insert(part="snippet,status", body=..., media_body=...)`.

- **Establecer miniatura personalizada:** La API soporta la subida de una imagen de miniatura con el endpoint `thumbnails.set`. Tras subir el video, se puede llamar a `youtube.thumbnails().set(videoId=<ID>, media_body=<archivo.jpg>)` para asignar la imagen generada en el paso 2. (Requiere que la cuenta de YouTube esté verificada para tener miniaturas personalizadas, y consumiría cuota de API).
- **Programar la publicación (Schedule):** YouTube permite programar la fecha/hora en que un video privado se hará público. Esto se logra al subir, indicando en el campo `status` del video la hora de publicación: se sube como `private` y se especifica `publishAt="YYYY-MM-DDThh:mm:ssZ"` en formato ISO 8601 ¹⁸. Por ejemplo: `publishAt="2025-05-15T13:00:00.000Z"` para programar la publicación el 15 de mayo de 2025 a las 13:00 UTC. Esta funcionalidad está soportada vía API (se consigue definiendo `status.privacyStatus="private"` junto con `status.publishAt` en la llamada a `videos.insert`) ¹⁹.
- **Añadir metadatos avanzados:** Además del título y descripción, la API permite añadir:
 - **Etiquetas (tags):** en el `snippet.tags` del video.
 - **Categoría:** `snippet.categoryId` (por ejemplo, 20 es Gaming).
 - **Playlists:** si se quiere agregar el video a una lista de reproducción, se usa `playlistItems.insert` con el ID de la playlist y del video.
- **Monetización y otros ajustes:** Algunas configuraciones (como monetización, licencia, audiencia "made for kids") también pueden ajustarse mediante campos en `status` o `audience`.
- **Subir subtítulos (captions):** La API de YouTube también permite subir pistas de subtítulos para un video mediante `captions.insert` ²⁰. Esto sería útil si generamos subtítulos offline (sección 4) y queremos agregarlos al video en YouTube de forma automatizada (como subtítulos opcionales que el usuario puede activar). Se puede subir un archivo .srt o .vtt indicando el idioma (por ejemplo, `es` para Español, `en` para Inglés) y asociándolo al videoId.
- **Otras acciones:** Borrar videos (`videos.delete`), actualizar algún campo después de subir (`videos.update`), comentar o responder comentarios, etc., también son posibles, aunque para nuestro caso quizás no se necesiten en el flujo principal.

Bibliotecas recomendadas: - *Google APIs Client Library (Python)* – simplifica la autenticación OAuth2 y las llamadas REST a la YouTube Data API. - *Pytube* – es una librería Python de terceros centrada en descargas de YouTube (no para subida). - *youtube-api* (JavaScript/Node) – para integraciones en un stack MERN por ejemplo, pero en nuestro caso usamos Python para la lógica server.

Cuotas y consideraciones: La API de YouTube tiene un sistema de cuota por unidad. Subir un video (`videos.insert`) cuesta 1600 unidades de cuota, establecer thumbnail 50, subtítulos 400, etc. Hay un límite diario (por defecto 10,000 unidades/día) que conviene vigilar ²¹. Si el flujo publica muchos videos al día, podría requerir solicitar aumento de cuota. Además, ciertas operaciones requieren que la cuenta de YouTube tenga privilegios (por ejemplo, miniaturas y programación requieren que el canal esté verificado y con buen standing, lo mismo para más de 10 videos subidos al día).

Ejemplo de implementación en Python: Utilizando la librería oficial, un pseudocódigo podría ser:

```

import google_auth_oauthlib.flow, googleapiclient.discovery

# Autenticación OAuth2
flow = google_auth_oauthlib.flow.InstalledAppFlow.from_client_secrets_file(
    "client_secrets.json", scopes=["https://www.googleapis.com/auth/
youtube.upload"]
)
credentials = flow.run_console() # o run_local_server()

youtube = googleapiclient.discovery.build("youtube", "v3",
credentials=credentials)

# Preparar metadatos del video
request_body = {
    "snippet": {
        "title": "Mi Video Editado",
        "description": "Descripción del video...",
        "tags": ["tag1", "tag2"],
        "categoryId": "24"
    },
    "status": {
        "privacyStatus": "private",
        "publishAt": "2025-05-15T13:00:00Z"
    }
}

# Subir video
request = youtube.videos().insert(
    part="snippet,status",
    body=request_body,
    media_body="video_editado.mp4"
)
response = request.execute()
video_id = response["id"]

# Subir miniatura personalizada
youtube.thumbnails().set(
    videoId=video_id,
    media_body="miniatura.jpg"
).execute()

```

Este script realiza autenticación, sube el video con los metadatos y lo programa, luego asigna la miniatura. De igual forma se podrían añadir los subtítulos generados con otra llamada API. Una vez configurado esto, **toda la publicación es hands-free**: el video quedará en el canal, con su título/descripción, miniatura y subtítulos, listo para estrenarse en la fecha programada.

4. Generación Automática de Subtítulos (Transcripción y Traducción)

Incluir subtítulos en los videos tiene múltiples beneficios: mejora la accesibilidad, retención de audiencia (muchos usuarios ven videos sin sonido) y permite llegar a espectadores de otros idiomas si se ofrecen subtítulos traducidos. Automatizar la creación de subtítulos en español (idioma original del contenido) y su posible traducción a otro idioma es viable gracias a avanzados modelos de reconocimiento de voz:

- **Transcripción de voz a texto en español (ASR):** La herramienta destacada actualmente es **OpenAI Whisper**. Se trata de un modelo de *Reconocimiento Automático de Voz* entrenado con cientos de miles de horas de audio multilingüe. Whisper es capaz de transcribir con *alta precisión* el habla en español a texto, incluso agregando puntuación y acentos correctamente ²². A diferencia de sistemas antiguos, Whisper entiende el contexto para producir frases coherentes, no solo palabras sueltas ²³. Es open-source y se puede ejecutar localmente (requiere una GPU si se usa el modelo grande para más velocidad, aunque también funciona en CPU más lentamente). Existe también un API de OpenAI para Whisper (servicio en la nube) que simplifica su uso sin infraestructura, con un costo aproximado de \$0.006/minuto.
- **Implementación:** Usar la biblioteca `whisper` de OpenAI (o su variante optimizada `whisper.cpp` para CPU) es sencillo:

```
import whisper
model = whisper.load_model("medium") # elegir tamaño de modelo
result = model.transcribe("video_editado.mp4", language="es")
text = result["text"] # transcripción completa
segments = result["segments"] # segmentos con tiempos para subtítulos
```

Esto produce segmentos con sus timestamps, que luego podemos formatear como .srt o .vtt. La precisión suele superar 95%, dependiendo del audio ²². Herramientas como **WhisperX** van un paso más allá e incorporan alineación precisa de palabras y reconocimiento de múltiples hablantes (diarización).

- **Alternativas de transcripción:** APIs en la nube como **Google Cloud Speech-to-Text**, **Amazon Transcribe** o **Azure Cognitive Services Speech** también soportan español latino y europeo. Por ejemplo, Google STT soporta más de 125 idiomas ²⁴ con alta fidelidad. La ventaja de estos servicios es su escalabilidad y posiblemente mejor manejo de audio ruidoso; la desventaja es el costo por minuto (aunque algunos ofrecen ciertos minutos gratuitos mensuales) y la necesidad de conexión. Dado que Whisper es gratuito localmente y muy preciso, muchos creadores lo están adoptando.
- **Traducción de subtítulos:** Una vez obtenida la transcripción en español, el siguiente paso opcional es traducirla a otro idioma (por ej. inglés) para generar una pista de subtítulos traducidos. Aquí se pueden emplear:
- **El propio Whisper:** Whisper tiene un modo de *traducción*, pero actualmente solo traduce **hacia el inglés** cualquier idioma de origen ²⁵. Es decir, puede escuchar audio en español y directamente generar texto en inglés, en lugar de español. Si el objetivo es obtener subtítulos en inglés, se podría ejecutar Whisper dos veces: una en modo transcribir a español, otra en modo

traducir a inglés. Sin embargo, la traducción de Whisper puede no ser tan fina como la de un modelo entrenado específicamente para traducir texto.

- **APIs de traducción especializadas:** Por ejemplo, **Google Translate API**, **DeepL API** o similares. Tomar el guion en español y traducirlo a inglés, o a otros idiomas deseados, de forma automática. DeepL tiende a tener traducciones de muy buena calidad. Estas APIs tienen coste por carácter traducido, pero para subtítulos (que no son larguísimos) puede ser manejable.
- **Modelos Open Source:** Como NLLB de Meta, o incluso usar un modelo grande (GPT) para traducir. Si no se quiere depender de servicios externos, hay modelos multilingües que pueden correr localmente para traducción, aunque implican complejidad.
- **Generación de archivos de subtítulos:** Con la transcripción segmentada (tiempos de inicios y finales de cada frase hablada), se puede construir un archivo .srt. Ejemplo de formato:

```
1
00:00:00,000 --> 00:00:03,000
Este es el inicio del stream, bienvenidos.

2
00:00:03,000 --> 00:00:06,500
[Música] Hoy jugaremos algo especial.
```

El `result["segments"]` de Whisper entrega segmentos con formato: `{ "start": 0.0, "end": 3.0, "text": " Este es el inicio del stream, bienvenidos." }`, etc. Solo hay que iterar y dar formato. Para la traducción, se podría mantener los mismos tiempos pero con el texto traducido.

- **Subir subtítulos a YouTube:** Como mencionamos, vía API es posible subir estos subtítulos. Alternativamente, se pueden cargar manualmente en YouTube Studio, pero dado que buscamos automatización completa, usar `captions.insert` con la pista en español y otra en inglés sería ideal. Esto requiere la autenticación OAuth con el scope apropiado y el video ya subido para asociarlos.

Ejemplo de pipeline de subtítulos automatizado:

1. Tras la edición del video, extraer el audio (por ejemplo, usar `ffmpeg` para obtener un `.wav` o usar directamente el video en Whisper).
2. Correr Whisper localmente para obtener la transcripción en español. Obtener también la versión traducida a inglés (sea con Whisper o enviando el texto a una API de traducción).
3. Formatear dos archivos: `subtitulos_es.srt` y `subtitles_en.srt`.
4. Usar la API de YouTube para subir ambas pistas de subtítulo al video correspondiente, indicando el lenguaje (`es` y `en`) y marcando la traducción apropiadamente (YouTube permite indicar si un caption es traducción de otro).
5. Opcional: también se puede *quemar* (embed) los subtítulos en el video durante la edición o mediante FFmpeg (para plataformas que no soporten subtítulos aparte, como redes sociales), pero en YouTube es mejor subirlos por separado para que el usuario elija activarlos.

Con este proceso, cada video publicado tendrá subtítulos en español listos (lo cual también ayuda al propio YouTube en la indexación del contenido para SEO) y potencialmente en inglés u otros idiomas, ampliando la audiencia alcanzable sin esfuerzo manual de transcripción/traducción.

5. Análisis de Rendimiento del Video (Título, Miniatura, CTR, Retención, SEO)

Una vez publicado el contenido, es fundamental **analizar su rendimiento** para extraer aprendizajes y mejorar continuamente. En este apartado consideramos cómo automatizar la recolección y visualización de métricas clave, así como qué herramientas pueden ayudar en el análisis de elementos como el título y la miniatura:

- **Métricas clave de YouTube:** A través de YouTube Analytics (accesible vía la **YouTube Analytics API** o desde YouTube Studio) podemos obtener:
 - **Vistas (views) y Tiempo de visualización** (watch time).
 - **Duración promedio de vista y % de video visto** (retención promedio).
 - **Métricas de engagement:** likes, comentarios, compartidos, crecimiento de suscriptores atribuible a ese video.
 - **Fuentes de tráfico:** qué porcentaje de vistas vino de búsqueda, de sugerencias, externo, etc.
 - **Retención de audiencia detallada:** YouTube proporciona curvas de retención (absoluta y relativa) a lo largo del video, identificando puntos de abandono.
- **Impresiones de miniatura y CTR:** cuántas veces se mostró la miniatura en pantallas de usuarios (impresiones) y qué porcentaje hizo clic (Click-Through Rate). *Nota:* Estas dos últimas son de las métricas más importantes para evaluar título+miniatura, pero curiosamente **no están disponibles en la API pública de YouTube** ²⁶ – solo se ven en YouTube Studio. Es decir, la API de Analytics no provee *Impressions* ni *Impression CTR* de los videos (posiblemente por políticas internas). Para obtener esos datos de forma automatizada, se podría usar una solución indirecta: por ejemplo, exportar los datos desde YouTube Studio a Google Data Studio o Sheets (YouTube permite descargar CSV de analytics en la interfaz web) o *web-scraping* autenticado de la página de YouTube Studio (no oficial). Pero es importante saber esa limitación de la API.
- **SEO (posicionamiento):** Aquí hablamos de qué tan bien el video se está encontrando por búsquedas. La API de Analytics permite obtener qué términos de búsqueda llevaron a vistas del video (en la sección de tráfico de búsqueda). Estas “keywords” pueden indicar si el título/etiquetas del video están alineados con lo que la gente busca. También se puede usar la API de *YouTube Search* para consultar la posición del video para ciertos términos, aunque eso es más complicado y no siempre permitido ampliamente.
- **YouTube Analytics API vs YouTube Studio:** Muchas de las métricas mencionadas están disponibles vía la API de Analytics. Por ejemplo, con la API uno puede obtener un reporte de retención que contiene métricas como *audienceWatchRatio* o *relativeRetentionPerformance* (retención relativa comparada con videos similares) ²⁷, entre otras. Los datos de CTR e impresiones, como se indicó, no están en la API ²⁶, por lo que habría que obtenerlos manualmente.
- **Herramientas de terceros (vidIQ, TubeBuddy, etc.):** Existen plataformas orientadas a YouTubers que ofrecen insights adicionales:
 - **vidIQ y TubeBuddy:** Son servicios con dashboards que evalúan el SEO de tus videos (puntuación de optimización, recomendaciones de tags, alertas de tendencias). Tienen APIs muy limitadas (generalmente no abiertas públicamente) pero podrían integrarse indirectamente vía sus exportaciones o CSV.

- **Social Blade:** Para ver crecimiento de suscriptores, proyecciones, etc., aunque a nivel de canal más que de video.
- **Grupo de discusión de miniaturas:** Una práctica interesante es hacer *A/B testing de miniaturas*. Actualmente YouTube está experimentando con permitir a algunos usuarios subir dos miniaturas y que la plataforma rote entre ellas para comparar rendimiento. Si eso no está disponible, TubeBuddy ofrecía un servicio similar (cambiando la miniatura a mitad de periodo y comparando métricas). Estas herramientas pueden ayudar a entender qué miniatura funciona mejor. Automatizar un A/B test podría ser complejo (tendría que un script cambiar la miniatura via API `thumbnails.set` cada X horas y registrar los resultados).
- **Análisis interno con Python:** Podemos construir nuestro propio análisis:
 - Usar la **YouTube Analytics API** para obtener datos diariamente de cada video (por ejemplo, vistas diarias durante la primera semana, retención promedio, etc.) y almacenarlos (en una base de datos o simplemente como JSON/CSV).
 - Con librerías como **pandas** y **matplotlib/plotly**, generar gráficos de tendencia: cómo evoluciona el CTR (si lo tenemos), cómo es la curva de retención (YouTube API permite obtener datos de retención en intervalos, aunque para más detalle quizás no granulares vía API).
 - Calcular tasas: por ejemplo, *ratio de like/dislike* (aunque YouTube ocultó los dislikes públicos, el creador puede verlos), *engagement rate* (comentarios por vista).
- **SEO keywords:** Podríamos usar la API de *Google Trends* o *YouTube suggest* para ver interés en ciertas palabras clave relacionadas al título, e incluso usar un modelo de lenguaje (GPT-4) para sugerir mejoras de título/descripción basado en el contenido del video (ya que tenemos la transcripción, se le podría pedir a un modelo que proponga 5 títulos alternativos con buen gancho).
- **APIs externas para SEO/CTR:** No hay muchas APIs abiertas para CTR de YouTube fuera de la plataforma, pero se puede emplear por ejemplo la API de **Google Analytics** si los videos están incrustados en una web propia para medir clics. Sin embargo, esto es enrevesado. Es más directo obtener lo que se pueda de YouTube y analizarlo.

En la práctica, se puede incorporar al flujo automatizado un módulo de **dashboard de analytics**: - Diariamente (o a intervalos) un script consulta la API de YouTube Analytics del canal para actualizar los datos de los últimos videos subidos (por ejemplo, últimos 10 videos). - Esos datos se guardan y se presentan en un panel (posiblemente construido con React, ver sección 7) mostrando tablas comparativas: cuál fue el rendimiento en 24h/48h de cada video (vistas, % retención, subs ganados). - Resaltar outliers: qué video tuvo CTR más alto (habrá que ingresar manualmente ese dato hasta no tenerlo por API), qué título generó más tráfico de búsqueda. - Incluso integrar con Google Sheets o Data Studio si se prefiere una visualización ya hecha.

Herramientas de YouTube Studio: YouTube Studio en sí ofrece una *API web interna* (no pública) que es la que la interfaz usa. Algunos desarrolladores han logrado usarla para sacar datos no disponibles en la API oficial (como la lista de comentarios con sentimientos, etc.). Esto requeriría emular peticiones autenticadas y puede romper si cambian. No es la vía más limpia, pero es posible.

Por último, para el análisis de SEO externo: - Usar **Google Trends** para ver interés en el tema del video a lo largo del tiempo. - Ver **rankings**: por ejemplo buscar en YouTube el tema principal y comprobar si el video aparece en resultados (posible con la API de búsqueda de YouTube). Esto se podría automatizar para ver si tras optimizar metadata el video sube de posición.

Conclusión de análisis: Con todas estas herramientas, el creador podría recibir informes automáticos. Ej: cada semana, un script podría enviar un email o generar un reporte con un resumen: *"Tu video X alcanzó 10k vistas (5% más que el anterior en el mismo periodo), retención media 45% hasta el minuto 5. El CTR de la miniatura fue 4.1%. Las principales búsquedas que trajeron tráfico: 'juego X final boss', 'highlight streamer Y'. Considera mejorar la miniatura o el inicio del video para incrementar la retención."* Gran parte de estos datos pueden obtenerse automáticamente (salvo CTR exacto vía API), permitiendo al creador tomar decisiones basadas en datos sin tener que revisar manualmente cada apartado de YouTube Studio.

6. Automatización de Adobe Premiere Pro y Photoshop via Scripts

Dado que en este flujo se utilizan Adobe Premiere Pro para la edición de video y Photoshop para diseño gráfico (miniaturas, overlays, etc.), es importante destacar cómo podemos **interactuar con estas aplicaciones de forma automatizada**. Si bien Adobe no está pensado para ser controlado externamente de forma sencilla, sí existen *scripting* y *APIs* que permiten cierto grado de automatización:

- **Adobe Premiere Pro: Extenscript y Pymiere:** Premiere Pro ofrece un entorno de script interno basado en **ExtendScript** (un dialecto de JavaScript similar a ES3) que puede ejecutar acciones dentro de Premiere: crear secuencias, insertar clips en la línea de tiempo, aplicar transiciones, exportar medios, etc. Tradicionalmente uno escribe un `.jsx` y lo ejecuta en Premiere. Para integrarlo con Python, existe un proyecto llamado **Pymiere** ²⁸, el cual proporciona una capa de Python para controlar Premiere. Pymiere funciona instalando una extensión en Premiere (*Pymiere Link*) que abre un canal HTTP local; la librería Python envía comandos ExtendScript a esa extensión para que Premiere los ejecute ²⁹ ³⁰. En esencia, **Pymiere es un wrapper de ExtendScript** que permite llamar directamente objetos y funciones de Premiere con sintaxis Python, convirtiéndolas internamente a código ExtendScript ²⁹. Por ejemplo, uno puede hacer en Python:

```
import pymiere
project = pymiere.objects.app.project # obtener proyecto abierto
seq = project.activeSequence
seq.name = "Nueva Secuencia Editada"
```

Esto renombraría la secuencia activa. Se pueden realizar operaciones complejas: agregar clips (`project.importFiles()` y luego `seq.videoTracks[n].insertClip(...)`), ajustar capas de texto en gráficos esenciales, marcar puntos de entrada/salida, etc., prácticamente cualquier cosa soportada por la API de scripting de Premiere ³¹. Hay que tener Premiere abierto para que funcione, no corre "headless" ³². En nuestro flujo, podríamos usarlo para, por ejemplo, tras descargar un VOD, abrir Premiere, cargar una plantilla de proyecto, insertar el VOD, aplicar recortes predefinidos (o incluso calculados, como quitar secciones de inactividad utilizando datos de volumen del audio), y luego mandar a exportar la secuencia final como MP4. Todo sin clicks manuales.

Limitaciones: Configurar Pymiere requiere instalar la extensión ZXP en Premiere, lo cual es un paso manual inicial. Además, las operaciones de edición complejas requieren conocer bien la estructura de objetos de Premiere. Aún así, Pymiere es potente para integrar Premiere en pipelines automatizados, preferible a métodos rudimentarios como simular teclas o ratón.

- **Adobe Photoshop:** Photoshop también soporta ExtendScript (JavaScript) y otras formas de scripting (VBScript en Windows, AppleScript en Mac). No obstante, Adobe no provee Python

nativo para Photoshop. Aquí podemos usar la biblioteca **Photoshop Python API** (disponible en PyPI) que se basa en controlar Photoshop a través de **COM** en Windows ³³. Es decir, vía la interfaz de automatización OLE/COM de Photoshop, uno puede abrir la aplicación, crear documentos, manipular capas, etc., usando Python. Esta biblioteca expone objetos Python equivalentes a los de scripting de Photoshop. Por ejemplo:

```
import photoshop.api as ps
app = ps.Application()           # conecta con Photoshop (debe estar
abierto, aunque se puede lanzar)
doc = app.open(r"C:\template.psd") # abre un PSD existente
layer = doc.artLayers["Titulo"]   # supongamos hay una capa de texto
llamada "Titulo"
layer.textItem.contents = "Nuevo Texto de Título"
doc.save()
```

Este snippet abriría un PSD de plantilla y cambiaría el texto de la capa "Titulo". Podemos asimismo insertar imágenes (`doc.ArtLayers.Add()`) luego `doc.Paste()` con una imagen copiada, o usar `app.load()`, aplicar filtros, fusionar capas, exportar para web, etc. **Casi cualquier acción disponible en Photoshop se puede scriptar**. De hecho, muchas agencias usan scripting para tareas repetitivas como generar lotes de banners cambiando textos o colores.

Para nuestro caso de miniaturas, podríamos tener un PSD base con ciertos elementos y usar Python para:

- Colocar la imagen de fondo generada o el frame del video.
- Insertar la foto recortada del creador.
- Editar el texto del título del video en la capa de texto.
- Aplicar estilos predefinidos (sombras, bordes) – estos ya estarían en la plantilla de capa, solo actualizamos el contenido.
- Guardar la imagen final en JPG.

Esto puede hacerse en segundos automáticamente. La Photoshop API Python facilita no tener que escribir JavaScript ExtendScript manualmente; internamente ejecuta JavaScript en Photoshop por nosotros, vía COM.

- **Adobe APIs oficiales en la nube:** Adobe tiene un servicio cloud llamado Photoshop API (parte de Adobe I/O) donde se pueden enviar imágenes para, por ejemplo, recortar sujeto, transferir estilo, etc ³⁴. Sin embargo, este es un servicio web que requiere suscripción enterprise; para nuestro flujo local, lo más práctico es el enfoque de scripting local.
- **Automatización de exportación/render:** Tanto Premiere como Photoshop pueden ser controlados para exportar resultados. En Premiere, tras montar la secuencia, se puede enviar la secuencia a Adobe Media Encoder (AME) para que renderice en segundo plano. Incluso AME se puede manejar vía línea de comandos para renderizar un proyecto o secuencia específica (usando archivos .pproj y nombre de secuencia). Photoshop, por su lado, puede exportar a imágenes sin diálogos vía script.
- **Otras herramientas externas:** En casos donde la edición a realizar es muy sistemática, cabe preguntarse si se puede hacer *fuera* de Premiere con herramientas FOSS. Por ejemplo, recortar un video de 1h a los 10 min mejores podría hacerse con `ffmpeg` si se conocen los timestamps, sin abrir Premiere. No obstante, para ediciones complejas con múltiples pistas, efectos o simplemente por preferencia del editor, usar Premiere asegura más control. La automatización vía script acelera el flujo repetitivo (especialmente si hay que hacer muchos videos similares).

Recomendaciones finales: Usar Pymiere y Photoshop COM scripting conecta bien con el stack Python:

- Podemos tener un script Python que, tras descargar el VOD, lance Premiere (Pymiere puede incluso abrir la aplicación si no está corriendo ³²), ejecute las acciones de edición, espere a que Premiere termine de exportar, y luego continuar con el upload a YouTube. Todo orquestado en Python.
- Similarmente con Photoshop: tras generar las imágenes con IA, un script ajusta la plantilla y guarda la miniatura final.

En resumen, si bien Adobe no expone un simple REST API para sus apps de escritorio, sí existen métodos para integrarlas en pipelines automatizados. Esto permite, por ejemplo, crear un sistema donde con solo introducir la URL de un VOD y algunos parámetros, el resto del proceso (edición + thumb + publicación) ocurra con mínima intervención humana.

7. Frameworks y Herramientas para un Dashboard de Control (Python & React)

Para orquestar todo este flujo de trabajo, es muy útil contar con una **interfaz central** donde el creador pueda supervisar y controlar cada etapa. La combinación de **Python (backend)** y **React (frontend)** en un entorno Windows ofrece mucha flexibilidad para construir un dashboard o aplicación web local. A continuación, discutimos la arquitectura y herramientas para ello:

- **Backend en Python:** La lógica pesada (descargas, llamadas a APIs, ejecución de IA, control de Adobe, etc.) residirá en el lado de Python. Se puede implementar un servidor web REST con frameworks ligeros como **Flask** o **FastAPI**:
- **Flask:** Muy sencillo de usar, ideal para exponer unas cuantas rutas (endpoints) que al ser invocadas disparen procesos. Por ejemplo, un `POST /process_video` que reciba la URL del VOD y devuelva un ID de tarea.
- **FastAPI:** Ofrece estructura similar a Flask pero con tipado y documentación automática, útil si el proyecto crece. Soporta WebSockets de forma fácil, lo que podría ser útil para enviar actualizaciones en tiempo real al frontend (p. ej., progreso de una descarga).
- **Django:** Podría ser usado si se requiere un proyecto más grande con base de datos, autenticaciones, etc., pero para una herramienta de uso individual podría ser excesivo. No obstante, Django Rest Framework podría servir si se planea exponer una API más compleja.

En Python también manejaremos probablemente **procesos en background**. Cuando el usuario en el panel dé clic a "Procesar nuevo video", el servidor Python puede iniciar un hilo o proceso separado que ejecute las funciones de descarga, edición, subida, etc., para no bloquear las respuestas web. Para esto:

- Podemos usar hilos/async simples dado que el número de tareas no será enorme.
- O integrar un **sistema de cola** como **Celery** con un broker (RabbitMQ/Redis) para manejar tareas en segundo plano robustamente. Celery podría ejecutar tarea de "procesar video X" y luego podremos consultar su estado.
- Incluso una alternativa más simple: usar **APScheduler** para programar tareas (por ejemplo, chequeos periódicos de analytics, o publicaciones diferidas si queremos manejarlo del lado nuestro en vez de YouTube).

La comunicación de Python con Adobe apps (Premiere/Photoshop) y con las otras APIs ya se describió. Python se encargará de coordinar eso y mantener el estado de qué está haciendo (descargando, editando, etc.).

- **Frontend en React:** El panel de control web puede ser una aplicación **React** (creada con Create React App, Next.js, Vite, etc.) que ofrezca las siguientes vistas:

- **Dashboard principal:** listar los videos procesados, en progreso y completados, con su estatus (por ejemplo: "Descargando VOD...", "Editando...", "Publicado en YouTube <link>"). Aquí se pueden ver logs básicos o porcentajes de progreso.
- **Formulario de nueva tarea:** campos para ingresar la URL del VOD, seleccionar opciones (por ejemplo, segmentos a recortar, idioma de subtítulos a traducir, título deseado para YouTube, etc.). Al enviar, hace una llamada POST al backend para iniciar proceso.
- **Vista de analíticas:** gráficas y tablas mostrando el rendimiento de los últimos videos, tal como se habló en sección 5. Esto implicaría que el backend exponga datos de analytics (por ej., un endpoint GET `/analytics?video_id=XYZ` que devuelva JSON de vistas, watchtime, etc., que React grafica).
- **Configuración:** quizás para almacenar credenciales (de Google API, de Twitch/Kick if needed), o templates de títulos/descripciones, etc.

React se comunica con el backend Python mediante **fetch/XHR** para solicitar acciones y datos. Para actualizaciones en vivo (p. ej., progreso de descarga al 50%), se pueden usar varias técnicas: - Polling periódico: el frontend pregunta cada 5 segundos por el estado de la tarea. - WebSockets: el backend (con FastAPI por ejemplo) envía mensajes en tiempo real cuando hay nuevos log updates, y el frontend los recibe para actualizar la UI inmediatamente.

En cuanto a diseño, se pueden usar librerías UI como **Material-UI (MUI)** o **Ant Design** para tener componentes ya estilizados y un layout agradable. Esto ahorra tiempo en crear botones, tablas, barras de progreso, etc.

- **Integración y despliegue:** Si es para uso personal/local, se puede ejecutar el React dev server para desarrollo y luego hacer un build estático y servirlo desde Flask (por ejemplo, colocando los archivos estáticos y teniendo Flask sirviendo `/` con `index.html`). Alternativamente, puede ser una app separada donde el React front se hospeda con `npm start` y el Python por separado – para un entorno local no hay problema con eso. Si se deseara desplegar remotamente (ej. en una VM o servicio en la nube) para controlar desde cualquier lugar, habría que considerar la seguridad (autenticación en el panel, etc.).
- **Dashboards ya hechos:** Existe la posibilidad de utilizar frameworks que combinan Python y frontend más fácilmente, por ejemplo **Plotly Dash** o **Streamlit** para las analíticas. Sin embargo, para el control de procesos, React + API REST ofrece más control. Otra alternativa visual es usar herramientas de automatización tipo **n8n** (que tiene una interfaz de workflows pre-hecha), pero dado que se requiere mucha personalización e integración con Adobe local, es mejor la solución a medida.
- **Ejemplos existentes:** El workflow publicado por n8n ³⁵ demuestra que es viable automatizar YouTube con metadata generado por IA, aunque en su caso todo ocurre sin una interfaz custom (n8n provee la GUI de nodos). En nuestro caso, construiremos la interfaz con React para mayor flexibilidad.

En pocas palabras, utilizar **Python como orquestador** nos da acceso a todas las bibliotecas necesarias (descarga, IA, APIs) en un solo lugar, y **React** nos permite crear una interfaz de usuario amigable para monitorear y parametrizar el proceso. Este tipo de arquitectura (frontend separado del backend) también hace más fácil depurar y mantener cada lado.

Características adicionales potenciales del dashboard:

- **Notificaciones:** Integrar con algún servicio de notificación (correo, Telegram, Discord webhook) para avisar cuando un proceso termine o si hubo algún error.
- **Multi-plataforma:** Tal vez en el futuro, además de subir a YouTube, se podría automatizar subir a otras plataformas (Facebook Video, Instagram TV, etc., que también tienen APIs). El panel podría permitir escoger destinos adicionales.
- **Control de versiones:** Llevar registro de qué cambios se hicieron automáticamente, por ejemplo, qué título se asignó, para luego ver si se quiere editar manualmente algo.

Con todo lo anterior, el creador tendría un sistema integral donde con unos pocos clics inicia la producción de un video editado y subido, y luego puede ver cómo le va. La mayor parte del *trabajo duro* (descargar, cortar, renderizar, subtitar, publicar) ocurre de forma automatizada gracias a la conjunción de herramientas que hemos detallado.

Referencias: Las herramientas y métodos descritos se basan en documentación oficial y proyectos de la comunidad. Por ejemplo, la API de YouTube permite programar videos estableciendo `privacyStatus` y `publishAt` ¹⁸; yt-dlp soporta descargas de múltiples sitios incluyendo Twitch ²; la generación de miniaturas con IA está facilitando lograr diseños atractivos rápidamente ¹¹; Whisper de OpenAI ofrece transcripciones precisas en español ²²; y proyectos como Pymiere habilitan la automatización de Premiere Pro vía Python ²⁹. Con estas bases, es posible construir un flujo de trabajo optimizado de punta a punta para creadores de contenido modernos.

- 1 2 **yt-dlp – downloader of videos from YouTube and other sites — Debian Neuroscience Package Repository**
<https://neuro.debian.net/pkgs/yt-dlp.html>
- 3 **Programmatically downloading VODS with oauth access - API - Twitch Developer Forums**
<https://discuss.dev.twitch.com/t/programmatically-downloading-vods-with-oauth-access/62605>
- 4 **GitHub - ihabunek/twitch-dl: CLI tool for downloading videos from Twitch.**
<https://github.com/ihabunek/twitch-dl>
- 5 6 8 **How you can download kick vods : r/KickStreaming**
https://www.reddit.com/r/KickStreaming/comments/14fv85p/how_you_can_download_kick_vods/
- 7 **GitHub - jonathanhecl/kick-vod-downloader: This is a tool to download the VOD from kick.com**
<https://github.com/jonathanhecl/kick-vod-downloader>
- 9 10 **GitHub - Likhithsai2580/yt_thumbnail_creator: Creating Thumbnails for Everyone, Everywhere**
https://github.com/Likhithsai2580/yt_thumbnail_creator
- 11 12 13 **Fliki - Turn text into videos with AI voices**
<https://fliki.ai/es/features/creador-de-miniaturas-con-ia>
- 14 15 33 **photoshop-python-api · PyPI**
<https://pypi.org/project/photoshop-python-api/>
- 16 17 **Upload a Video | YouTube Data API | Google for Developers**
https://developers.google.com/youtube/v3/guides/uploading_a_video
- 18 19 **Can I schedule YouTube videos for release via the API? And other 'limited' feature questions - Stack Overflow**
<https://stackoverflow.com/questions/20025814/can-i-schedule-youtube-videos-for-release-via-the-api-and-other-limited-featu>
- 20 21 **Captions: insert | YouTube Data API | Google for Developers**
<https://developers.google.com/youtube/v3/docs/captions/insert>
- 22 23 25 **Cómo generar subtítulos y transcripciones automáticas con Inteligencia Artificial**
<https://www.josemariafernandez.es/2024/01/como-generar-subtitulos-y-transcripciones-automaticas-con-inteligencia-artificial/>
- 24 **Speech-to-Text AI: speech recognition and transcription**
<https://cloud.google.com/speech-to-text>
- 26 **YouTube Analytics API - Impressions and Click Through Rate ...**
<https://issuetracker.google.com/issues/254665034>
- 27 **Youtube Analytics API v3 - retrieving relative audience retention**
<https://stackoverflow.com/questions/40345970/youtube-analytics-api-v3-retrieving-relative-audience-retention>
- 28 29 30 31 32 **qmasingarbe/pymiere: Python for Premiere pro - GitHub**
<https://github.com/qmasingarbe/pymiere>
- 34 **Adobe Photoshop API Overview**
<https://developer.adobe.com/photoshop/photoshop-api-docs/>
- 35 **Automated YouTube Video Scheduling & AI Metadata Generation | n8n workflow template**
<https://n8n.io/workflows/3900-automated-youtube-video-scheduling-and-ai-metadata-generation/>